Departamento de Computação



Disciplina: Sistemas Multimídia (CMP 1103)
Prof. Esp. Thalles Bruno Santos

Atividade Prática - Desenvolvimento de Aplicação de Streaming com GStreamer

Objetivo

Desenvolver uma aplicação utilizando C ou C++ e GStreamer para transmitir e receber fluxos de áudio por uma rede, explorando mecanismos de controle de latência, compensação de jitter e tratamento de perda de pacotes. A atividade deverá resultar em uma aplicação funcional, permitindo relacionar os conceitos estudados na disciplina ao desenvolvimento de software para transmissão multimídia.

Proposta

Desenvolva uma aplicação composta por dois componentes:

- Transmissor: responsável por obter uma fonte de áudio, realizar os processamentos necessários e transmitir os dados utilizando UDP/RTP.
- Receptor: responsável por receber os pacotes, reconstruir o fluxo de áudio e realizar sua reprodução.

A arquitetura das pipelines, a escolha dos elementos e a implementação deverão ser pesquisadas na documentação oficial do GStreamer.

Você poderá reaproveitar estruturas e funcionalidades desenvolvidas nas atividades anteriores, desde que implemente os novos requisitos.

Requisitos da aplicação

1. Transmissão de áudio

Implemente um componente capaz de:

- utilizar uma fonte de áudio, como microfone, arquivo ou sinal gerado pelo GStreamer;
- preparar o áudio para transmissão;
- encapsular o fluxo em pacotes RTP;
- transmitir os pacotes utilizando UDP;
- permitir configurar o endereço e a porta de destino.

2. Recepção de áudio

Implemente um componente capaz de:

- receber os pacotes transmitidos por UDP;
- identificar e processar os pacotes RTP;
- reconstruir o fluxo de áudio recebido;
- reproduzir o áudio em um dispositivo de saída;
- tratar erros básicos de inicialização e execução.

O transmissor e o receptor poderão ser executados no mesmo computador, utilizando o endereço 127.0.0.1, ou em computadores diferentes conectados à mesma rede.

3. Controle de jitter e latência

Evolua o componente receptor para incorporar mecanismos de compensação de jitter.

A aplicação deverá:

- utilizar um elemento de gerenciamento de jitter, como rtpjitterbuffer;
- permitir configurar diferentes valores de latência;
- executar testes com pelo menos três configurações;
- registrar o comportamento da reprodução em cada configuração.

Investigue como o buffer influencia a reprodução e quais são as limitações dessa abordagem.

4. Investigação de perda de pacotes

Pesquise mecanismos utilizados pelo GStreamer para identificar ou tratar pacotes RTP perdidos.

Implemente pelo menos uma funcionalidade relacionada ao diagnóstico de perdas, como:

- identificação de eventos de perda de pacotes;
- consulta a estatísticas do rtpjitterbuffer;
- registro de pacotes descartados ou recebidos fora do prazo.

Caso disponha de um ambiente que permita simular atrasos e perdas de pacotes, realize experimentos comparando o comportamento do receptor.

Não é necessário implementar um algoritmo próprio de recuperação de pacotes. O objetivo é investigar e utilizar os recursos disponibilizados pelo framework.

Diagramas das pipelines

Elabore diagramas representando as pipelines efetivamente implementadas.

Os diagramas deverão identificar:

- fontes de mídia;
- elementos de processamento;
- elementos responsáveis pelo encapsulamento RTP;
- componentes de transmissão e recepção UDP;
- mecanismos de gerenciamento de jitter;
- elementos de reprodução;
- principais parâmetros configurados.

Utilize Excalidraw, Draw.io ou ferramenta equivalente.

Os diagramas deverão corresponder ao código desenvolvido, permitindo compreender o fluxo de dados entre os componentes.

Demonstração

Ao concluir a implementação, demonstre diretamente ao professor:

1. A aplicação transmitindo e reproduzindo áudio.
2. A execução do transmissor e do receptor.
3. A alteração dos parâmetros de latência.
4. Os mecanismos implementados para identificação ou tratamento de perdas.
5. Os diagramas das pipelines.
6. Os principais trechos de código responsáveis pela construção e execução da solução.

Durante a demonstração, você deverá explicar as decisões técnicas adotadas e as dificuldades encontradas.

Entregáveis

A entrega deverá conter:

- Código-fonte completo, em C ou C++, dos componentes desenvolvidos.
- Arquivo README.md com instruções para compilação, configuração e execução.
- Diagramas das pipelines em Excalidraw, Draw.io ou formato equivalente.
- Registro dos experimentos realizados, incluindo parâmetros utilizados e resultados observados.
- Breve conclusão sobre o comportamento do streaming e os mecanismos de controle de jitter e perda de pacotes.

Documentação de apoio

Consulte os seguintes materiais oficiais do GStreamer durante o desenvolvimento.

Construção programática de pipelines

Basic Tutorial 2 - GStreamer Concepts -
https://gstreamer.freedesktop.org/documentation/tutorials/basic/concepts.html

Apresenta os conceitos necessários para criação, ligação e gerenciamento de elementos em aplicações C/C++.

Basic Tutorial 6 - Media Formats and Pad Capabilities -
https://gstreamer.freedesktop.org/documentation/tutorials/basic/media-formats-and-pad-capabilities.html

Aborda a negociação de formatos e capacidades entre elementos, importante para configurar corretamente os fluxos RTP.

Basic Tutorial 10 - GStreamer Tools -
https://gstreamer.freedesktop.org/documentation/tutorials/basic/gstreamer-tools.html

Apresenta ferramentas para investigar elementos e testar pipelines antes de implementá-las programaticamente.

Transmissão e recepção

UDP Source - udpsrc - https://gstreamer.freedesktop.org/documentation/udp/udpsrc.html

UDP Sink - udpsink - https://gstreamer.freedesktop.org/documentation/udp/udpsink.html

RTP - Elementos e plugins - https://gstreamer.freedesktop.org/documentation/rtp/

RTP e RTSP - Visão geral - https://gstreamer.freedesktop.org/documentation/additional/rtp.html

Controle de jitter e perda de pacotes

RTP Jitter Buffer - rtpjitterbuffer -
https://gstreamer.freedesktop.org/documentation/rtpmanager/rtpjitterbuffer.html

Apresenta propriedades relacionadas à latência, reorganização de pacotes e estatísticas de perdas e atrasos.

Orientações para o desenvolvimento

Antes de implementar os componentes em C/C++, utilize gst-inspect-1.0 para investigar os elementos disponíveis e gst-launch-1.0 para validar experimentalmente as pipelines.

Pesquise especialmente os elementos relacionados a UDP, RTP, conversão de áudio e gerenciamento de jitter.

A implementação final deverá construir e executar as pipelines programaticamente. A entrega exclusiva de comandos gst-launch-1.0 não atende ao objetivo da atividade.

Não serão fornecidas pipelines completas. A pesquisa, a seleção dos elementos e a justificativa das decisões fazem parte do desenvolvimento.

Critérios de avaliação

A avaliação considerará:

- funcionamento da transmissão e recepção de áudio;
- implementação correta das pipelines em C/C++;
- utilização dos mecanismos de controle de jitter;
- implementação de recursos de diagnóstico de perdas;
- organização e qualidade do código-fonte;
- coerência dos diagramas apresentados;
- capacidade de explicar e justificar as decisões técnicas.

A atividade deverá demonstrar a capacidade de desenvolver uma solução multimídia funcional, utilizando recursos de transmissão em rede e gerenciamento de fluxos contínuos.